

Лекция по курсу «ОС и СП» №3

План лекции.

1. Память. Виды. Классификация.
2. Управление памятью. Фиксированные, динамические и перемещаемые разделы.
3. Виртуальная память. Страничное, сегментное, сегментно-страничное распределение памяти.
4. Реестр Windows.
5. Настройка конфигурации аппаратных средств. Профили оборудования. Ресурсы ОС. Управление продуктивностью.

Цели лекции.

Разобраться в основных принципах управления памятью в ОС, ознакомиться с фиксированными, динамическими, и сегментированными видами распределения памяти. Ознакомиться с принципами конфигурации аппаратных средств, созданием профилей оборудования. Ознакомиться с ресурсами ОС, тем, как в ОС управлять продуктивностью.

Теоретическая часть лекции.

1. Память. Виды. Классификация.

Компьютерная память (устройство хранения информации, запоминающее устройство) — часть ЭВМ, физическое устройство или среда для хранения данных, используемая в вычислениях, в течение определённого времени. Память, как и центральный процессор, является неизменной частью компьютера с 1940-х. Память в вычислительных устройствах имеет иерархическую структуру и обычно предполагает использование нескольких запоминающих устройств, имеющих различные характеристики.

В персональных компьютерах «памятью» часто называют один из её видов — динамическая память с произвольным доступом (DRAM), — которая в настоящее время используется в качестве ОЗУ персонального компьютера.

Задачей компьютерной памяти является хранение в своих ячейках состояния внешнего воздействия, запись информации. Эти ячейки могут фиксировать самые разнообразные физические воздействия. Они функционально аналогичны обычному электромеханическому переключателю и информация в них записывается в виде двух чётко различимых состояний — 0 и 1 («выключено»/«включено»). **Специальные механизмы обеспечивают доступ (считывание, произвольное или последовательное) к состоянию этих ячеек.**

Процесс доступа к памяти разбит на разделённые во времени процессы — операцию записи (сленг. *прошивка*, в случае записи ПЗУ) и **операцию чтения**, во многих случаях эти операции происходят под управлением отдельного специализированного устройства — контроллера памяти.

Также различают **операцию стирания памяти** — занесение (запись) в ячейки памяти одинаковых значений, обычно 00₁₆ или FF₁₆.

Наиболее известные запоминающие устройства, используемые в персональных компьютерах: модули оперативной памяти (ОЗУ), жёсткие диски (винчестеры), дискеты (гибкие магнитные диски), CD- или DVD-диски, а также устройства флеш-памяти.

Функции памяти

Компьютерная память обеспечивает поддержку одной из функций современного компьютера, — способность длительного хранения информации. Вместе с центральным процессором запоминающее устройство являются ключевыми звеньями так называемой архитектуры фон Неймана, — принципа, заложенного в основу большинства современных компьютеров общего назначения.

Первые компьютеры использовали запоминающие устройства исключительно для хранения обрабатываемых данных. Их программы реализовывались на аппаратном уровне в виде жёстко заданных выполняемых последовательностей. Любое перепрограммирование требовало огромного объёма ручной работы по подготовке новой документации, перекоммутации, перестройки блоков и устройств и т.д. Использование архитектуры фон Неймана, предусматривающей хранение компьютерных программ и данных в общей памяти, коренным образом переменяло ситуацию.

Любая информация может быть измерена в битах и потому, независимо от того, на каких физических принципах и в какой системе счисления функционирует цифровой компьютер (двоичной, троичной, десятичной и т.п.), числа, текстовая информация, изображения, звук, видео и другие виды данных можно представить последовательностями битовых строк или двоичными числами. Это позволяет компьютеру манипулировать данными при условии достаточной ёмкости системы хранения (например, для хранения текста романа среднего размера необходимо около одного мегабайта).

К настоящему времени создано множество устройств, предназначенных для хранения данных, основанных на использовании самых разных физических эффектов. Универсального решения не существует, у каждого имеются свои достоинства и свои недостатки, поэтому компьютерные системы обычно оснащаются несколькими видами систем хранения, основные свойства которых обуславливают их использование и назначение.

Физические основы функционирования

В основе работы запоминающего устройства может лежать любой физический эффект, обеспечивающий приведение системы к двум или более устойчивым состояниям. В современной компьютерной технике часто используются физические свойства полупроводников, когда прохождение тока через полупроводник или его отсутствие трактуются как наличие логических сигналов 0 или 1. Устойчивые состояния, определяемые направлением намагниченности, позволяют использовать для хранения данных разнообразные магнитные материалы. Наличие или отсутствие заряда в конденсаторе также может быть положено в основу системы хранения. Отражение или рассеяние света от поверхности CD, DVD или Blu-ray-диска также позволяет хранить информацию.

Классификация типов памяти

Следует различать *классификацию памяти* и *классификацию запоминающих устройств* (ЗУ). Первая классифицирует память по *функциональности*, вторая же — по *технической реализации*. Здесь рассматривается первая — таким образом, в неё попадают как аппаратные виды памяти (реализуемые на ЗУ), так и *структуры данных*, реализуемые в большинстве случаев программно.

Доступные операции с данными

- Память только для чтения (*read-only memory, ROM*)
- Память для чтения/записи

Память на программируемых и перепрограммируемых ПЗУ (ППЗУ и ПППЗУ) не имеет общепринятого места в этой классификации. Её относят либо к подвиду памяти «только для чтения», либо выделяют в отдельный вид.

Также предлагается относить память к тому или иному виду по характерной частоте её перезаписи на практике: к RAM относить виды, в которых информация часто меняется в процессе работы, а к ROM — предназначенные для хранения относительно неизменных данных.

Метод доступа

- **Последовательный доступ** (англ. *sequential access memory, SAM*) — ячейки памяти выбираются (считываются) последовательно, одна за другой, в очередности их расположения. Вариант такой памяти — стековая память.

- **Произвольный доступ** (англ. *random access memory, RAM*) — вычислительное устройство может обратиться к произвольной ячейке памяти по любому адресу.

Организация хранения данных и алгоритмы доступа к ним повторяет классификацию структур данных:

- **Адресуемая память** — адресация осуществляется по местоположению данных.
- **Ассоциативная память** (англ. *associative memory, content-addressable memory, CAM*) — адресация осуществляется по содержанию данных, а не по их местоположению (память проверяет наличие ячейки с заданным содержимым, и, если таковая(ые) присутствует(ют), возвращает ее(их) адрес(а) или другие данные с ней(ними) ассоциированные).

- **Магазинная (стековая) память** (англ. *pushdown storage*) — реализация стека.
- **Матричная память** (англ. *matrix storage*) — ячейки памяти расположены так, что доступ к ним осуществляется по двум или более координатам.

- **Объектная память** (англ. *object storage*) — память, система управления которой ориентирована на хранение объектов. При этом каждый объект характеризуется типом и размером записи.

- **Семантическая память** (англ. *semantic storage*) — данные размещаются и списываются в соответствии с некоторой структурой понятийных признаков.

И др.

Назначение

- **Буферная память** (англ. *buffer storage*) — память, предназначенная для временного хранения данных при обмене ими между различными устройствами или программами.

- **Временная (промежуточная) память** (англ. *temporary (intermediate) storage*) — память для хранения промежуточных результатов обработки.

- **Кеш-память** (англ. *cache memory*) — часть архитектуры устройства или программного обеспечения, осуществляющая хранение часто используемых данных для предоставления их в более быстрый доступ, нежели кешируемая память.

- **Корректирующая память** (англ. *patch memory*) — часть памяти ЭВМ, предназначенная для хранения адресов неисправных ячеек основной памяти. Также используются термины *relocation table* и *remap table*.

- **Управляющая память** (англ. *control storage*) — память, содержащая управляющие программы или микропрограммы. Обычно реализуется в виде ПЗУ.

- **Разделяемая память или память коллективного доступа** (англ. *shared memory, shared access memory*) — память, доступная одновременно нескольким пользователям, процессам или процессорам.

И др.

Организация адресного пространства

- **Реальная или физическая память** (англ. *real (physical) memory*) — память, способ адресации которой соответствует физическому расположению её данных;

- **Виртуальная память** (англ. *virtual memory*) — память, способ адресации которой не отражает физического расположения её данных;

- **Оверлейная память** (англ. *overlayable storage*) — память, в которой присутствует несколько областей с одинаковыми адресами, из которых в каждый момент доступна только одна.

Удалённость и доступность для процессора

- **Первичная память** (сверхоперативная, СОЗУ) — доступна процессору без какого-либо обращения к внешним устройствам. Данная память отличается крайне малым временем доступа и тем, что не адресуема для программиста.

- ✓ регистры процессора (*процессорная или регистровая память*) — регистры, расположенные непосредственно в арифметически-логическом устройстве (АЛУ);

- ✓ кэш процессора — кэш, используемый процессором для уменьшения среднего времени доступа к компьютерной памяти. Разделяется на несколько уровней, различающихся скоростью и объёмом (например, L1, L2, L3).

- **Вторичная память** — доступна процессору путём прямой адресации через шину адреса (*адресуемая память*). Таким образом доступна *оперативная память* (память, предназначенная для хранения текущих данных и выполняемых программ) и *порты ввода-вывода* (специальные адреса, через обращение к которым реализовано взаимодействие с прочей аппаратурой).

- **Третичная память** — доступна только путём нетривиальной последовательности действий. Сюда входят все виды *внешней памяти* — доступной через устройства ввода-вывода. Взаимодействие с третичной памятью ведётся по определённым правилам (протоколам) и требует присутствия в памяти соответствующих программ. Программы, обеспечивающие минимально необходимое взаимодействие, помещаются в ПЗУ, входящее во вторичную память (у PC-совместимых ПК — это ПЗУ BIOS).

Положение структур данных, расположенных в основной памяти, в этой классификации неоднозначно. Как правило, их вообще в неё не включают, выполняя классификацию с привязкой к традиционно используемым видам ЗУ.

Доступность техническими средствами

- **Непосредственно управляемая** (*оперативно доступная*) память (англ. *on-line storage*) — память, непосредственно доступная в данный момент.

- **Автономная память**, Архив (англ. *off-line storage*) — память, доступ к которой требует внешних действий — например, вставку оператором в архивного носителя с указанным программой идентификатором.

- **Полуавтономная память** (англ. *nearline storage*) — то же, что автономная, но физическое перемещение носителей осуществляется роботом по команде системы, то есть не требует присутствия оператора.

Прочие термины

- **Многоблочная память** (англ. *multibank memory*) — вид оперативной памяти, организованной из нескольких независимых блоков, допускающих одновременное обращение к ним, что повышает её пропускную способность. Часто употребляется термин «интерлив» (калька с англ. *interleave* — перемежать) и может встречаться в документации некоторых фирм «многоканальная память» (англ. *multichannel*).

- **Память со встроенной логикой** (англ. *logic-in-memory*) — вид памяти, содержащий встроенные средства логической обработки (преобразования) данных, например, их масштабирования, преобразования кодов, наложения полей и др.

- **Многовходовая память** (англ. *multiport storage memory*) — устройство памяти, допускающее независимое обращение с нескольких направлений (входов), причём обслуживание запросов производится в порядке их приоритета.

- **Многоуровневая память** (англ. *multilevel memory*) — организация памяти, состоящая из нескольких уровней запоминающих устройств с различными характеристиками и рассматриваемая со стороны пользователей как единое целое. Для многоуровневой памяти

характерна страничная организация, обеспечивающая «прозрачность» обмена данными между ЗУ разных уровней.

- **Память параллельного действия** (англ. *parallel storage*) — вид памяти, в которой все области поиска могут быть доступны одновременно.
- **Страничная память** (англ. *page memory*) — память, разбитая на одинаковые области — страницы. Операции записи-чтения на них осуществляются путём переключения страниц контроллером памяти.

2. Управление памятью. Фиксированные, динамические и перемещаемые разделы.

Управление памятью

Память является важнейшим ресурсом, требующим тщательного управления со стороны мультипрограммной операционной системы.

Распределению подлежит вся оперативная память, не занятая операционной системой. Обычно ОС располагается в самых младших адресах, однако может занимать и самые старшие адреса.

Функциями ОС по управлению памятью являются: отслеживание свободной и занятой памяти, выделение памяти процессам и освобождение памяти при завершении процессов, вытеснение процессов из оперативной памяти на диск, когда размеры основной памяти не достаточны для размещения в ней всех процессов, и возвращение их в оперативную память, когда в ней освобождается место, а также настройка адресов программы на конкретную область физической памяти.

Типы адресов

Для идентификации переменных и команд используются символьные имена (метки), виртуальные адреса и физические адреса (рисунок 1).

Символьные имена присваивает пользователь при написании программы на алгоритмическом языке или ассемблере.

Виртуальные адреса вырабатывает транслятор, переводящий программу на машинный язык. Так как во время трансляции в общем случае не известно, в какое место оперативной памяти будет загружена программа, то транслятор присваивает переменным и командам виртуальные (условные) адреса, обычно считая по умолчанию, что программа будет размещена, начиная с нулевого адреса. **Совокупность виртуальных адресов процесса называется виртуальным адресным пространством.** Каждый процесс имеет собственное виртуальное адресное пространство. Максимальный размер виртуального адресного пространства ограничивается разрядностью адреса, присущей данной архитектуре компьютера, и, как правило, не совпадает с объемом физической памяти, имеющимся в компьютере.

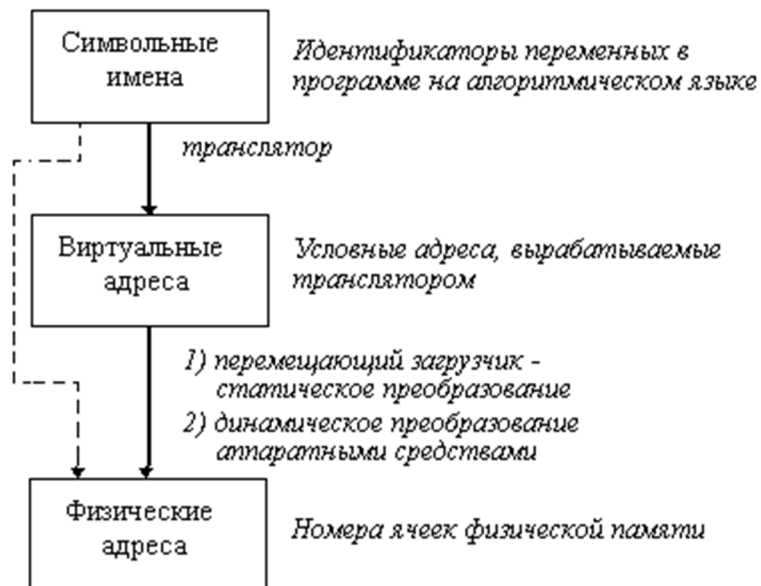


Рис. 1. Типы адресов

Физические адреса соответствуют номерам ячеек оперативной памяти, где в действительности расположены или будут расположены переменные и команды. **Переход от виртуальных адресов к физическим может осуществляться двумя способами. В первом случае замену виртуальных адресов на физические делает специальная системная программа - перемещающий загрузчик.** Перемещающий загрузчик на основании имеющихся у него исходных данных о начальном адресе физической памяти, в которую предстоит загружать программу, и информации, предоставленной транслятором об адресно-зависимых константах программы, выполняет загрузку программы, совмещая ее с заменой виртуальных адресов физическими.

Второй способ заключается в том, что программа загружается в память в неизменном виде в виртуальных адресах, при этом операционная система фиксирует смещение действительного расположения программного кода относительно виртуального адресного пространства. Во время выполнения программы при каждом обращении к оперативной памяти выполняется преобразование виртуального адреса в физический. Второй способ является более гибким, он допускает перемещение программы во время ее выполнения, в то время как перемещающий загрузчик жестко привязывает программу к первоначально выделенному ей участку памяти. Вместе с тем использование перемещающего загрузчика уменьшает накладные расходы, так как преобразование каждого виртуального адреса происходит только один раз во время загрузки, а во втором случае - каждый раз при обращении по данному адресу.

В некоторых случаях (обычно в специализированных системах), когда заранее точно известно, в какой области оперативной памяти будет выполняться программа, транслятор выдает исполняемый код сразу в физических адресах.

Методы распределения памяти без использования дискового пространства

Все методы управления памятью могут быть разделены на два класса: методы, которые используют перемещение процессов между оперативной памятью и диском, и методы, которые не делают этого (рисунок 2). Начнем с последнего, более простого класса методов.



Рис. 2. Классификация методов распределения памяти

Распределение памяти фиксированными разделами

Самым простым способом управления оперативной памятью является разделение ее на несколько разделов фиксированной величины. Это может быть выполнено вручную оператором во время старта системы или во время ее генерации. Очередная задача, поступившая на выполнение, помещается либо в общую очередь (рисунок 3.а), либо в очередь к некоторому разделу (рисунок 3.б).

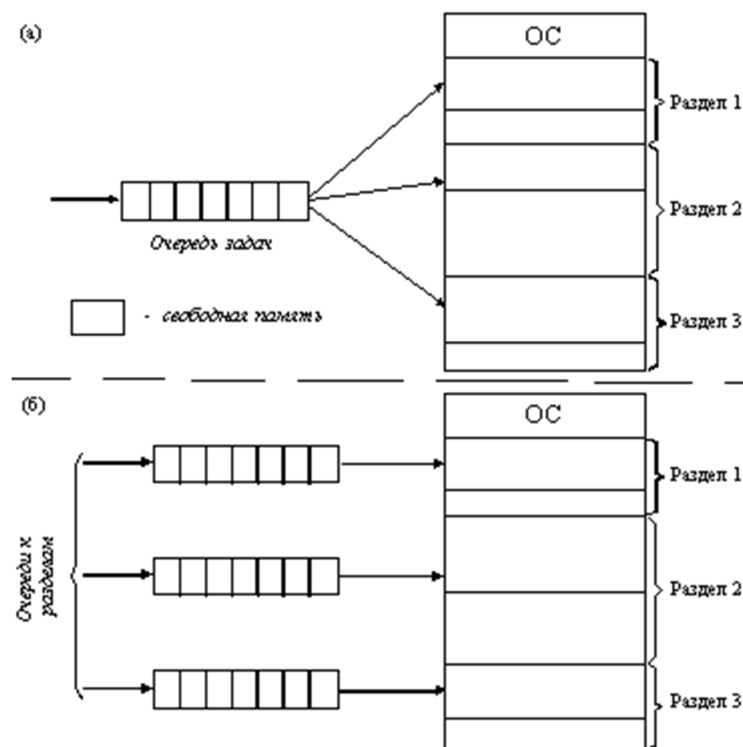


Рис. 3. Распределение памяти фиксированными разделами:
а - с общей очередью; б - с отдельными очередями

Подсистема управления памятью в этом случае выполняет следующие задачи:

- сравнивая размер программы, поступившей на выполнение, и свободных разделов, выбирает подходящий раздел,
- осуществляет загрузку программы и настройку адресов.

При очевидном преимуществе - простоте реализации - данный метод имеет существенный недостаток - жесткость. Так как в каждом разделе может выполняться только одна программа, то уровень мультипрограммирования заранее ограничен числом разделов не зависимо от того, какой размер имеют программы. Даже если программа имеет небольшой объем, она будет занимать

весь раздел, что приводит к неэффективному использованию памяти. С другой стороны, даже если объем оперативной памяти машины позволяет выполнить некоторую программу, разбиение памяти на разделы не позволяет сделать этого.

Распределение памяти разделами переменной величины

В этом случае память машины не делится заранее на разделы. Сначала вся память свободна. Каждой вновь поступающей задаче выделяется необходимая ей память. Если достаточный объем памяти отсутствует, то задача не принимается на выполнение и стоит в очереди. После завершения задачи память освобождается, и на это место может быть загружена другая задача. Таким образом, в произвольный момент времени оперативная память представляет собой случайную последовательность занятых и свободных участков (разделов) произвольного размера. На рисунке 4 показано состояние памяти в различные моменты времени при использовании динамического распределения. Так в момент t_0 в памяти находится только ОС, а к моменту t_1 память разделена между 5 задачами, причем задача П4, завершаясь, покидает память. На освободившееся после задачи П4 место загружается задача П6, поступившая в момент t_3 .

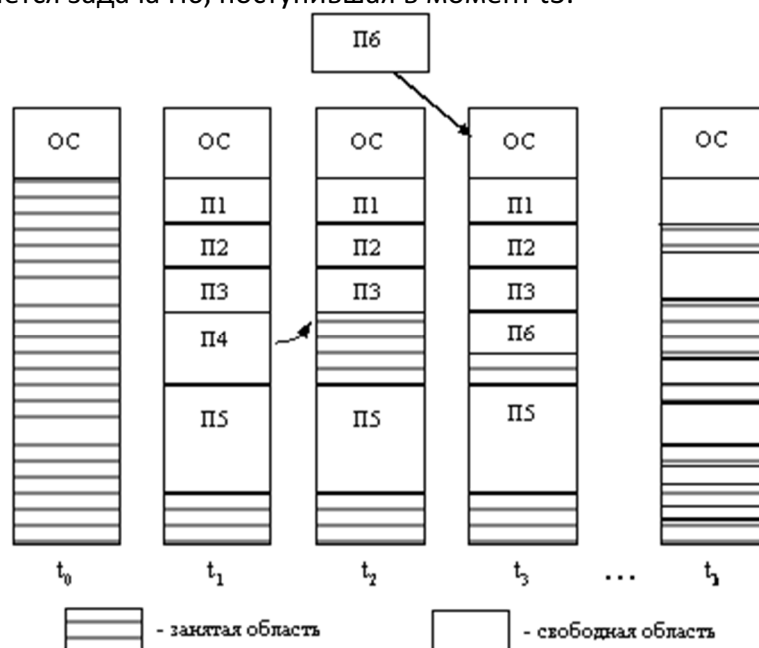


Рис. 4. Распределение памяти динамическими разделами

Задачами операционной системы при реализации данного метода управления памятью является:

- ведение таблиц свободных и занятых областей, в которых указываются начальные адреса и размеры участков памяти,
- при поступлении новой задачи - анализ запроса, просмотр таблицы свободных областей и выбор раздела, размер которого достаточен для размещения поступившей задачи,
- загрузка задачи в выделенный ей раздел и корректировка таблиц свободных и занятых областей,
- после завершения задачи корректировка таблиц свободных и занятых областей.

Программный код не перемещается во время выполнения, то есть может быть проведена единовременная настройка адресов посредством использования перемещающего загрузчика.

Выбор раздела для вновь поступившей задачи может осуществляться по разным правилам, таким, например, как "первый попавшийся раздел достаточного размера", или "раздел, имеющий наименьший достаточный размер", или "раздел, имеющий наибольший достаточный размер". Все эти правила имеют свои преимущества и недостатки.

По сравнению с методом распределения памяти фиксированными разделами данный метод обладает гораздо большей гибкостью, но ему присущ очень серьезный недостаток - *фрагментация*

памяти. Фрагментация - это наличие большого числа несмежных участков свободной памяти очень маленького размера (фрагментов). Настолько маленького, что ни одна из вновь поступающих программ не может поместиться ни в одном из участков, хотя суммарный объем фрагментов может составить значительную величину, намного превышающую требуемый объем памяти.

Перемещаемые разделы

Одним из методов борьбы с фрагментацией является перемещение всех занятых участков в сторону старших либо в сторону младших адресов, так, чтобы вся свободная память образовывала единую свободную область (рисунок 5). В дополнение к функциям, которые выполняет ОС при распределении памяти переменными разделами, в данном случае она должна еще время от времени копировать содержимое разделов из одного места памяти в другое, корректируя таблицы свободных и занятых областей. Эта процедура называется "сжатием". Сжатие может выполняться либо при каждом завершении задачи, либо только тогда, когда для вновь поступившей задачи нет свободного раздела достаточного размера. В первом случае требуется меньше вычислительной работы при корректировке таблиц, а во втором - реже выполняется процедура сжатия. Так как программы перемещаются по оперативной памяти в ходе своего выполнения, то преобразование адресов из виртуальной формы в физическую должно выполняться динамическим способом.

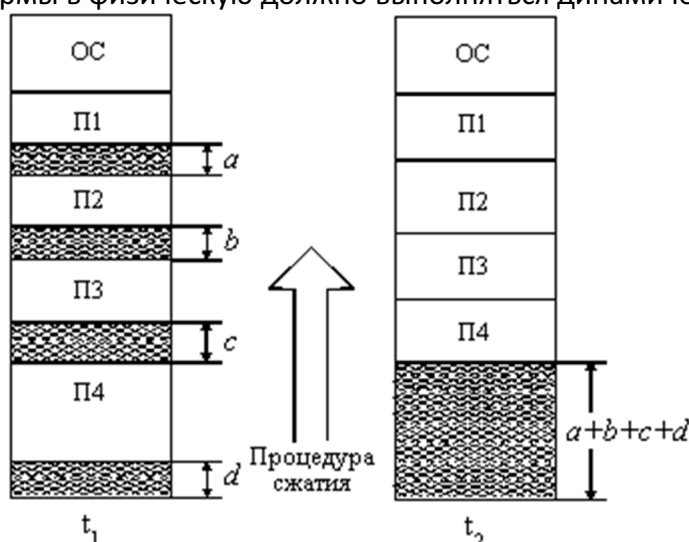


Рис. 5. Распределение памяти перемещаемыми разделами

Хотя процедура сжатия и приводит к более эффективному использованию памяти, она может потребовать значительного времени, что часто перевешивает преимущества данного метода.

3. Виртуальная память. Страничное, сегментное, сегментно-страничное распределение памяти.

Методы распределения памяти с использованием дискового пространства

Понятие виртуальной памяти

Уже достаточно давно пользователи столкнулись с проблемой размещения в памяти программ, размер которых превышал имеющуюся в наличии свободную память. Решением было разбиение программы на части, называемые *оверлеями*. 0-ой оверлей начинал выполняться первым. Когда он заканчивал свое выполнение, он вызывал другой оверлей. Все оверлеи хранились на диске и перемещались между памятью и диском средствами операционной системы. Однако разбиение программы на части и планирование их загрузки в оперативную память должен был осуществлять программист.

Развитие методов организации вычислительного процесса в этом направлении привело к появлению метода, известного под названием *виртуальная память*. Виртуальным называется ресурс, который пользователю или пользовательской программе представляется обладающим

свойствами, которыми он в действительности не обладает. Так, например, пользователю может быть предоставлена виртуальная оперативная память, размер которой превосходит всю имеющуюся в системе реальную оперативную память. Пользователь пишет программы так, как будто в его распоряжении имеется однородная оперативная память большого объема, но в действительности все данные, используемые программой, хранятся на одном или нескольких разнородных запоминающих устройствах, обычно на дисках, и при необходимости частями отображаются в реальную память.

Таким образом, виртуальная память - это совокупность программно-аппаратных средств, позволяющих пользователям писать программы, размер которых превосходит имеющуюся оперативную память; для этого виртуальная память решает следующие задачи:

- размещает данные в запоминающих устройствах разного типа, например, часть программы в оперативной памяти, а часть на диске;
- перемещает по мере необходимости данные между запоминающими устройствами разного типа, например, подгружает нужную часть программы с диска в оперативную память;
- преобразует виртуальные адреса в физические.

Все эти действия выполняются *автоматически*, без участия программиста, то есть механизм виртуальной памяти является прозрачным по отношению к пользователю.

Наиболее распространенными реализациями виртуальной памяти является страничное, сегментное и странично-сегментное распределение памяти, а также свопинг.

Страничное распределение

На рисунке 6 показана схема страничного распределения памяти. Виртуальное адресное пространство каждого процесса делится на части одинакового, фиксированного для данной системы размера, называемые виртуальными страницами. В общем случае размер виртуального адресного пространства не является кратным размеру страницы, поэтому последняя страница каждого процесса дополняется фиктивной областью.

Вся оперативная память машины также делится на части такого же размера, называемые физическими страницами (или блоками).

Размер страницы обычно выбирается равным степени двойки: 512, 1024 и т.д., это позволяет упростить механизм преобразования адресов.

При загрузке процесса часть его виртуальных страниц помещается в оперативную память, а остальные - на диск. Смежные виртуальные страницы не обязательно располагаются в смежных физических страницах. При загрузке операционная система создает для каждого процесса информационную структуру - таблицу страниц, в которой устанавливается соответствие между номерами виртуальных и физических страниц для страниц, загруженных в оперативную память, или делается отметка о том, что виртуальная страница выгружена на диск. Кроме того, в таблице страниц содержится управляющая информация, такая как признак модификации страницы, признак невыгружаемости (выгрузка некоторых страниц может быть запрещена), признак обращения к странице (используется для подсчета числа обращений за определенный период времени) и другие данные, формируемые и используемые механизмом виртуальной памяти.

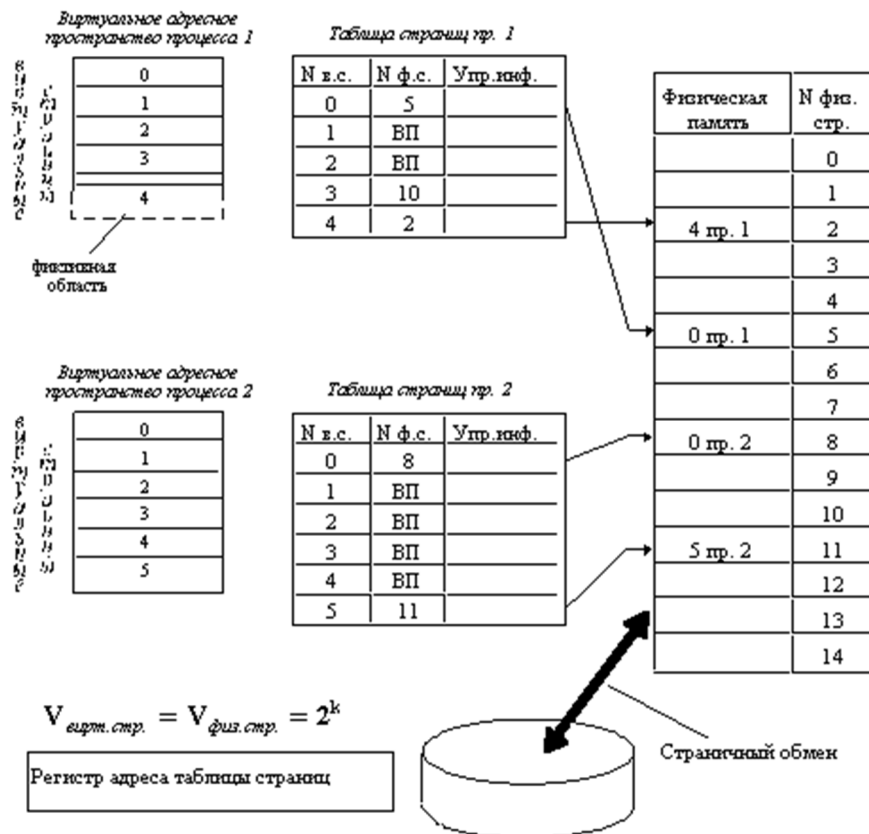


Рис. 6. Страничное распределение памяти

При активизации очередного процесса в специальный регистр процессора загружается адрес таблицы страниц данного процесса.

При каждом обращении к памяти происходит чтение из таблицы страниц информации о виртуальной странице, к которой произошло обращение. Если данная виртуальная страница находится в оперативной памяти, то выполняется преобразование виртуального адреса в физический. Если же нужная виртуальная страница в данный момент выгружена на диск, то происходит так называемое страничное прерывание. Выполняющийся процесс переводится в состояние ожидания, и активизируется другой процесс из очереди готовых. Параллельно программа обработки страничного прерывания находит на диске требуемую виртуальную страницу и пытается загрузить ее в оперативную память. Если в памяти имеется свободная физическая страница, то загрузка выполняется немедленно, если же свободных страниц нет, то решается вопрос, какую страницу следует выгрузить из оперативной памяти.

В данной ситуации может быть использовано много разных критериев выбора, наиболее популярные из них следующие:

- дольше всего не использовавшаяся страница,
- первая попавшаяся страница,
- страница, к которой в последнее время было меньше всего обращений.

В некоторых системах используется понятие рабочего множества страниц. Рабочее множество определяется для каждого процесса и представляет собой перечень наиболее часто используемых страниц, которые должны постоянно находиться в оперативной памяти и поэтому не подлежат выгрузке.

После того, как выбрана страница, которая должна покинуть оперативную память, анализируется ее признак модификации (из таблицы страниц). Если выталкиваемая страница с момента загрузки была модифицирована, то ее новая версия должна быть переписана на диск. Если нет, то она может быть просто уничтожена, то есть соответствующая физическая страница объявляется свободной.

Рассмотрим механизм преобразования виртуального адреса в физический при страничной организации памяти (рисунок 7).

Виртуальный адрес при страничном распределении может быть представлен в виде пары (p , s), где p - номер виртуальной страницы процесса (нумерация страниц начинается с 0), а s - смещение в пределах виртуальной страницы. Учитывая, что размер страницы равен 2^k , смещение s может быть получено простым отделением k младших разрядов в двоичной записи виртуального адреса. Оставшиеся старшие разряды представляют собой двоичную запись номера страницы p .

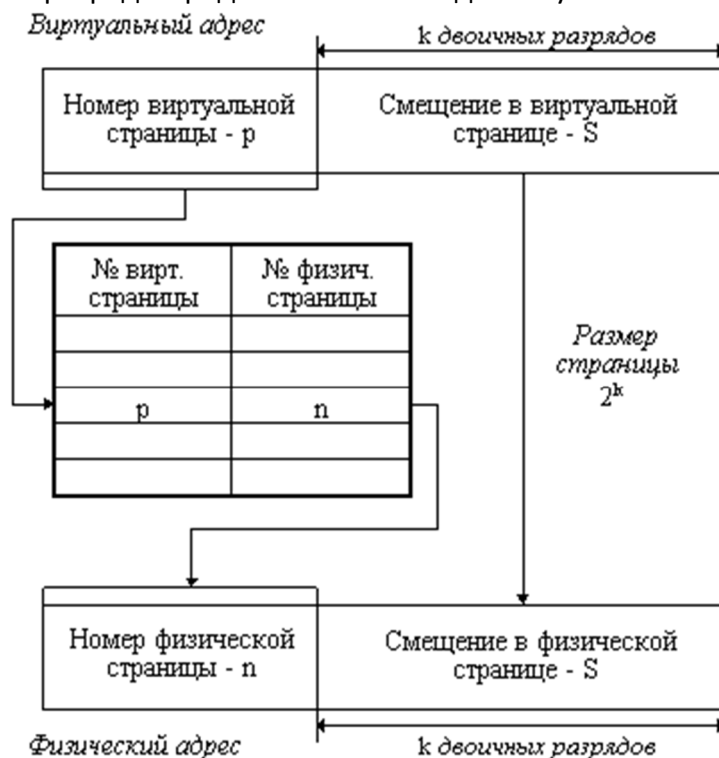


Рис. 7. Механизм преобразования виртуального адреса в физический при страничной организации памяти

При каждом обращении к оперативной памяти аппаратными средствами выполняются следующие действия:

1. на основании начального адреса таблицы страниц (содержимое регистра адреса таблицы страниц), номера виртуальной страницы (старшие разряды виртуального адреса) и длины записи в таблице страниц (системная константа) определяется адрес нужной записи в таблице,
2. из этой записи извлекается номер физической страницы,
3. к номеру физической страницы присоединяется смещение (младшие разряды виртуального адреса).

Использование в пункте (3) того факта, что размер страницы равен степени 2, позволяет применить операцию конкатенации (присоединения) вместо более длительной операции сложения, что уменьшает время получения физического адреса, а значит повышает производительность компьютера.

На производительность системы со страничной организацией памяти влияют временные затраты, связанные с обработкой страничных прерываний и преобразованием виртуального адреса в физический. При часто возникающих страничных прерываниях система может тратить большую часть времени впустую, на свопинг страниц. Чтобы уменьшить частоту страничных прерываний, следовало бы увеличивать размер страницы. Кроме того, увеличение размера страницы уменьшает размер таблицы страниц, а значит уменьшает затраты памяти. С другой стороны, если страница велика, значит велика и фиктивная область в последней виртуальной странице каждой программы. В среднем на каждой программе теряется половина объема страницы, что в сумме при большой странице может составить существенную величину. Время преобразования виртуального адреса в физический в значительной степени определяется временем доступа к таблице страниц. В связи с этим таблицу страниц стремятся размещать в "быстрых" запоминающих устройствах. Это может

быть, например, набор специальных регистров или память, использующая для уменьшения времени доступа ассоциативный поиск и кэширование данных.

Страничное распределение памяти может быть реализовано в упрощенном варианте, без выгрузки страниц на диск. В этом случае все виртуальные страницы всех процессов постоянно находятся в оперативной памяти. Такой вариант страничной организации хотя и не предоставляет пользователю виртуальной памяти, но почти исключает фрагментацию за счет того, что программа может загружаться в несмежные области, а также того, что при загрузке виртуальных страниц никогда не образуется остатков.

Сегментное распределение

При страничной организации виртуальное адресное пространство процесса делится механически на равные части. Это не позволяет дифференцировать способы доступа к разным частям программы (сегментам), а это свойство часто бывает очень полезным. Например, можно запретить обращаться с операциями записи и чтения в кодовый сегмент программы, а для сегмента данных разрешить только чтение. Кроме того, разбиение программы на "осмысленные" части делает принципиально возможным разделение одного сегмента несколькими процессами. Например, если два процесса используют одну и ту же математическую подпрограмму, то в оперативную память может быть загружена только одна копия этой подпрограммы.

Рассмотрим, каким образом сегментное распределение памяти реализует эти возможности (рисунок 8). Виртуальное адресное пространство процесса делится на сегменты, размер которых определяется программистом с учетом смыслового значения содержащейся в них информации. Отдельный сегмент может представлять собой подпрограмму, массив данных и т.п. Иногда сегментация программы выполняется по умолчанию компилятором.

При загрузке процесса часть сегментов помещается в оперативную память (при этом для каждого из этих сегментов операционная система подыскивает подходящий участок свободной памяти), а часть сегментов размещается в дисковой памяти. Сегменты одной программы могут занимать в оперативной памяти несмежные участки. Во время загрузки система создает таблицу сегментов процесса (аналогичную таблице страниц), в которой для каждого сегмента указывается начальный физический адрес сегмента в оперативной памяти, размер сегмента, правила доступа, признак модификации, признак обращения к данному сегменту за последний интервал времени и некоторая другая информация. Если виртуальные адресные пространства нескольких процессов включают один и тот же сегмент, то в таблицах сегментов этих процессов делаются ссылки на один и тот же участок оперативной памяти, в который данный сегмент загружается в единственном экземпляре.

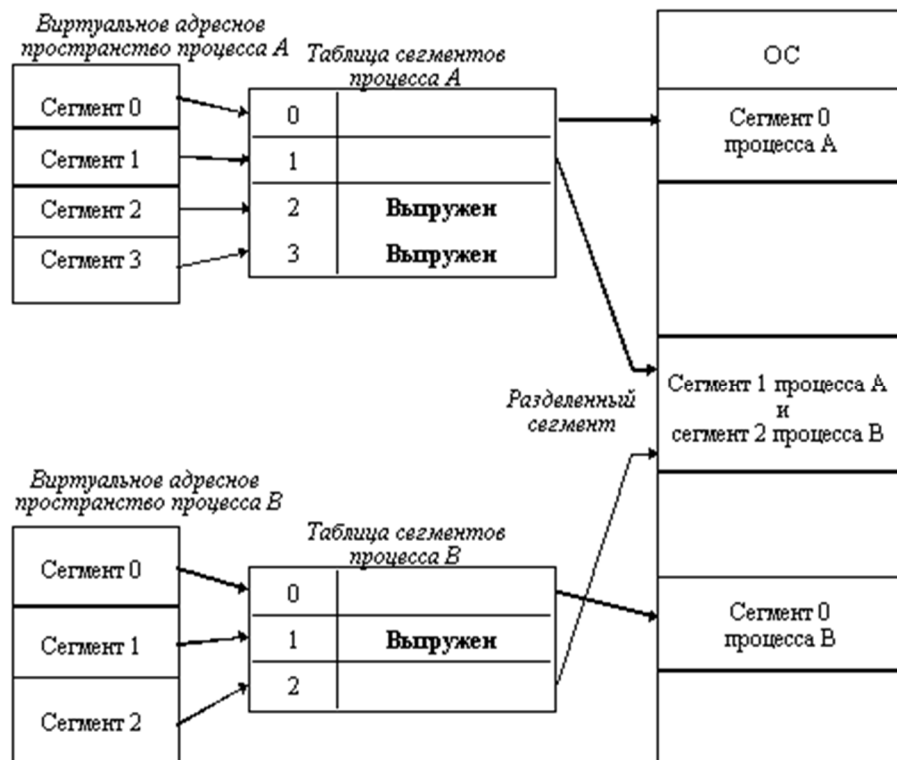


Рис. 8. Распределение памяти сегментами

Система с сегментной организацией функционирует аналогично системе со страничной организацией: время от времени происходят прерывания, связанные с отсутствием нужных сегментов в памяти, при необходимости освобождения памяти некоторые сегменты выгружаются, при каждом обращении к оперативной памяти выполняется преобразование виртуального адреса в физический. Кроме того, при обращении к памяти проверяется, разрешен ли доступ требуемого типа к данному сегменту.

Виртуальный адрес при сегментной организации памяти может быть представлен парой (g, s), где g - номер сегмента, а s - смещение в сегменте. Физический адрес получается путем сложения начального физического адреса сегмента, найденного в таблице сегментов по номеру g, и смещения s.

Недостатком данного метода распределения памяти является фрагментация на уровне сегментов и более медленное по сравнению со страничной организацией преобразование адреса.

Странично-сегментное распределение

Как видно из названия, данный метод представляет собой комбинацию страничного и сегментного распределения памяти и, вследствие этого, сочетает в себе достоинства обоих подходов. Виртуальное пространство процесса делится на сегменты, а каждый сегмент в свою очередь делится на виртуальные страницы, которые нумеруются в пределах сегмента. Оперативная память делится на физические страницы. Загрузка процесса выполняется операционной системой постранично, при этом часть страниц размещается в оперативной памяти, а часть на диске. Для каждого сегмента создается своя таблица страниц, структура которой полностью совпадает со структурой таблицы страниц, используемой при страничном распределении. Для каждого процесса создается таблица сегментов, в которой указываются адреса таблиц страниц для всех сегментов данного процесса. Адрес таблицы сегментов загружается в специальный регистр процессора, когда активизируется соответствующий процесс. На рисунке 9 показана схема преобразования виртуального адреса в физический для данного метода.

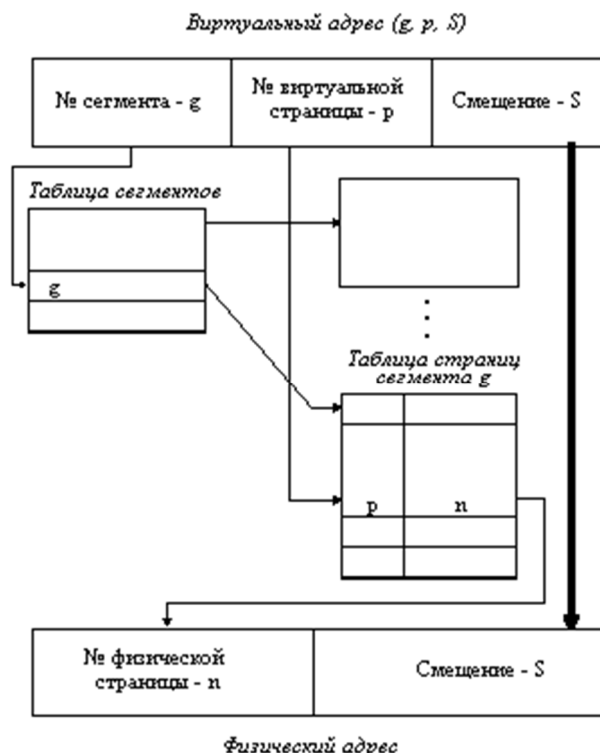


Рис. 9. Схема преобразования виртуального адреса в физический для сегментно-страничной организации памяти

Свопинг

Разновидностью виртуальной памяти является свопинг.

На рисунке 10 показан график зависимости коэффициента загрузки процессора в зависимости от числа одновременно выполняемых процессов и доли времени, проводимого этими процессами в состоянии ожидания ввода-вывода.

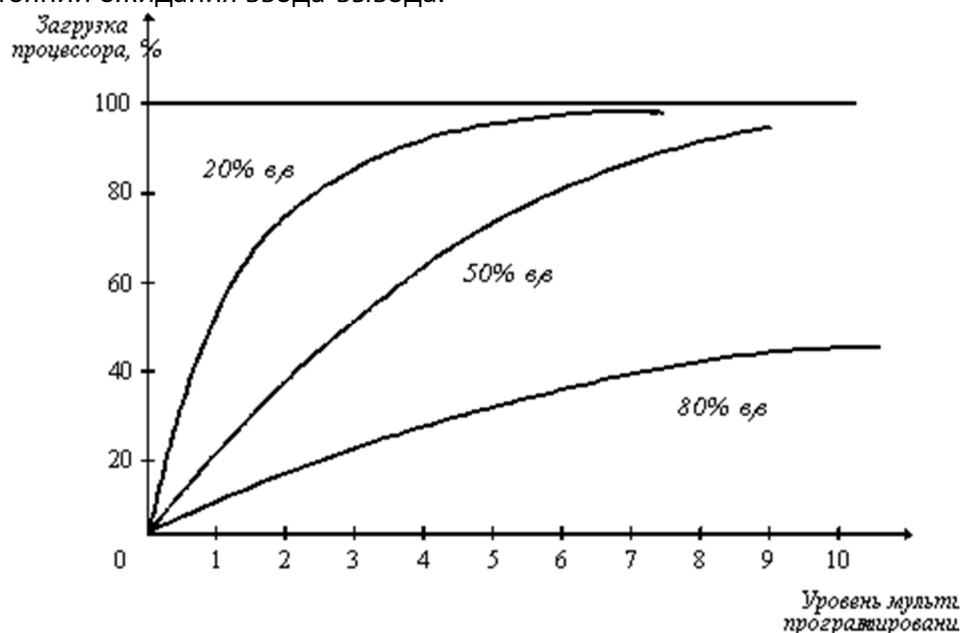


Рис. 10. Зависимость загрузки процессора от числа задач и интенсивности ввода-вывода

Из рисунка видно, что для загрузки процессора на 90% достаточно всего трех счетных задач. Однако для того, чтобы обеспечить такую же загрузку интерактивными задачами, выполняющими интенсивный ввод-вывод, потребуются десятки таких задач. Необходимым условием для выполнения задачи является загрузка ее в оперативную память, объем которой ограничен. В этих условиях был предложен метод организации вычислительного процесса, называемый свопингом. В соответствии с этим методом некоторые процессы (обычно находящиеся в состоянии ожидания) временно выгружаются на диск. Планировщик операционной системы не исключает их из своего

рассмотрения, и при наступлении условий активизации некоторого процесса, находящегося в области свопинга на диске, этот процесс перемещается в оперативную память. Если свободного места в оперативной памяти не хватает, то выгружается другой процесс.

При свопинге, в отличие от рассмотренных ранее методов реализации виртуальной памяти, процесс перемещается между памятью и диском целиком, то есть в течение некоторого времени процесс может полностью отсутствовать в оперативной памяти. Существуют различные алгоритмы выбора процессов на загрузку и выгрузку, а также различные способы выделения оперативной и дисковой памяти загружаемому процессу.

Дополнительно

Иерархия запоминающих устройств. Принцип кэширования данных

Память вычислительной машины представляет собой иерархию запоминающих устройств (внутренние регистры процессора, различные типы сверхоперативной и оперативной памяти, диски, ленты), отличающихся средним временем доступа и стоимостью хранения данных в расчете на один бит (рисунок 11). Пользователю хотелось бы иметь и недорогую и быструю память. Кэш-память представляет некоторое компромиссное решение этой проблемы.

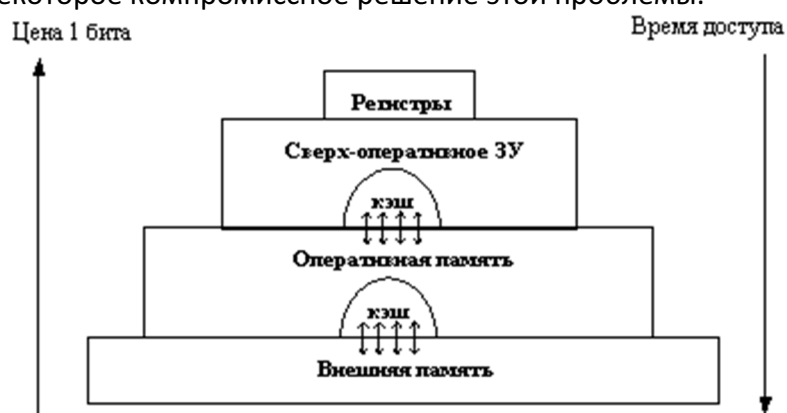
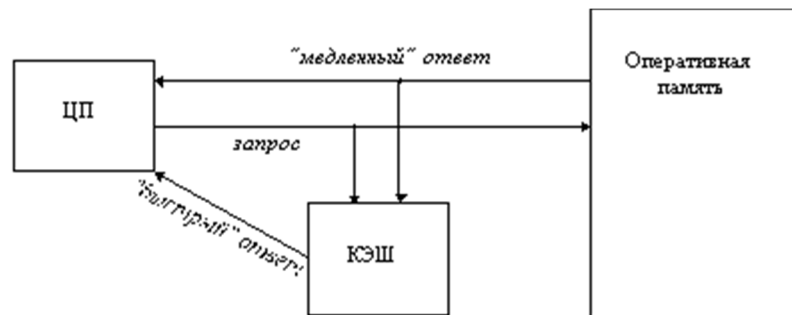


Рис. 11. Иерархия ЗУ

Кэш-память - это способ организации совместного функционирования двух типов запоминающих устройств, отличающихся временем доступа и стоимостью хранения данных, который позволяет уменьшить среднее время доступа к данным за счет динамического копирования в "быстрое" ЗУ наиболее часто используемой информации из "медленного" ЗУ.

Кэш-памятью часто называют не только способ организации работы двух типов запоминающих устройств, но и одно из устройств - "быстрое" ЗУ. Оно стоит дороже и, как правило, имеет сравнительно небольшой объем. Важно, что механизм кэш-памяти является прозрачным для пользователя, который не должен сообщать никакой информации об интенсивности использования данных и не должен никак участвовать в перемещении данных из ЗУ одного типа в ЗУ другого типа, все это делается автоматически системными средствами.

Рассмотрим частный случай использования кэш-памяти для уменьшения среднего времени доступа к данным, хранящимся в оперативной памяти. Для этого между процессором и оперативной памятью помещается быстрое ЗУ, называемое просто кэш-памятью (рисунок 12). В качестве такового может быть использована, например, ассоциативная память. Содержимое кэш-памяти представляет собой совокупность записей обо всех загруженных в нее элементах данных. Каждая запись об элементе данных включает в себя адрес, который этот элемент данных имеет в оперативной памяти, и управляющую информацию: признак модификации и признак обращения к данным за некоторый последний период времени.



Структура кэш-памяти

Адрес данных в ОП	Данные	Управл. информация	
		Бит модиф.	Бит обрац.

Рис. 12. Кэш-память

В системах, оснащенных кэш-памятью, каждый запрос к оперативной памяти выполняется в соответствии со следующим алгоритмом:

1. Просматривается содержимое кэш-памяти с целью определения, не находятся ли нужные данные в кэш-памяти; кэш-память не является адресуемой, поэтому поиск нужных данных осуществляется по содержимому - значению поля "адрес в оперативной памяти", взятому из запроса.
2. Если данные обнаруживаются в кэш-памяти, то они считываются из нее, и результат передается в процессор.
3. Если нужных данных нет, то они вместе со своим адресом копируются из оперативной памяти в кэш-память, и результат выполнения запроса передается в процессор. При копировании данных может оказаться, что в кэш-памяти нет свободного места, тогда выбираются данные, к которым в последний период было меньше всего обращений, для вытеснения из кэш-памяти. Если вытесняемые данные были модифицированы за время нахождения в кэш-памяти, то они переписываются в оперативную память. Если же эти данные не были модифицированы, то их место в кэш-памяти объявляется свободным.

На практике в кэш-память считывается не один элемент данных, к которому произошло обращение, а целый блок данных, это увеличивает вероятность так называемого "попадания в кэш", то есть нахождения нужных данных в кэш-памяти.

Покажем, как среднее время доступа к данным зависит от вероятности попадания в кэш. Пусть имеется основное запоминающее устройство со средним временем доступа к данным t_1 и кэш-память, имеющая время доступа t_2 , очевидно, что $t_2 < t_1$. Обозначим через t среднее время доступа к данным в системе с кэш-памятью, а через p - вероятность попадания в кэш. По формуле полной вероятности имеем:

$$t = t_1(1 - p) + t_2(p)$$

Из нее видно, что среднее время доступа к данным в системе с кэш-памятью линейно зависит от вероятности попадания в кэш и изменяется от среднего времени доступа в основное ЗУ (при $p=0$) до среднего времени доступа непосредственно в кэш-память (при $p=1$).

В реальных системах вероятность попадания в кэш составляет примерно 0,9. Высокое значение вероятности нахождения данных в кэш-памяти связано с наличием у данных объективных свойств: пространственной и временной локальности.

- **Пространственная локальность.** Если произошло обращение по некоторому адресу, то с высокой степенью вероятности в ближайшее время произойдет обращение к соседним адресам.

- *Временная локальность.* Если произошло обращение по некоторому адресу, то следующее обращение по этому же адресу с большой вероятностью произойдет в ближайшее время.

Все предыдущие рассуждения справедливы и для других пар запоминающих устройств, например, для оперативной памяти и внешней памяти. В этом случае уменьшается среднее время доступа к данным, расположенным на диске, и роль кэш-памяти выполняет буфер в оперативной памяти.

4. Реестр Windows.

Описание реестра

Реестр Windows (англ. *Windows Registry*), или **системный реестр** — иерархически построенная база данных параметров и настроек в большинстве операционных систем Microsoft Windows.

Реестр содержит информацию и настройки для аппаратного обеспечения, программного обеспечения, профилей пользователей, предустановки. Большинство изменений в Панели управления, ассоциации файлов, системные политики, список установленного ПО фиксируются в реестре.

Реестр Windows был введён для упорядочения информации, хранившейся до этого во множестве INI-файлов, обеспечения единого механизма (API) записи-чтения настроек и избавления от проблем коротких имён, отсутствия разграничения прав доступа и медленного доступа к ini-файлам, хранящимся на файловой системе FAT16, имевшей серьёзные проблемы быстродействия при поиске файлов в директориях с большим их количеством. Со временем (окончательно — с появлением файловой системы NTFS) проблемы, решавшиеся реестром, исчезли, но реестр остался из-за обратной совместимости, и присутствует во всех версиях Windows, включая последнюю. Поскольку сейчас не существует реальных предпосылок для использования подобного механизма, Microsoft Windows — единственная (не считая ReactOS) операционная система из используемых сегодня, в которой используется механизм реестра операционной системы.

Реестр Windows 3.1

Сам реестр как древовидная иерархическая база данных (registration database — регистрационная база) впервые появился в Windows 3.1 (апрель 1992). Это был всего один двоичный файл, который назывался REG.DAT и хранился в каталоге C:\Windows\. Реестр Windows 3.1 имел только одну ветку HKEY_CLASSES_ROOT. Он служил для связи DDE, а позднее и OLE объектов.

Одновременно с появлением реестра в Windows 3.1 появилась программа REGEDIT.EXE для просмотра и редактирования реестра.

Первый реестр уже имел возможность импорта данных из *.REG файлов. В базовой поставке шёл файл SETUP.REG, содержащий данные по основным расширениям и типам файлов.

Реестр Windows 3.1 имел ограничение на максимальный размер файла REG.DAT — 64 Кбайт. Если реестр превышал этот размер, файл реестра (REG.DAT) приходилось удалять и собирать заново либо из *.REG файлов, либо вводить данные вручную.

Реестр Windows NT 3.1

Следующий шаг был сделан в Windows NT 3.1 (июль 1993). Произошёл отказ от устаревших файлов MS-DOS: AUTOEXEC.BAT и CONFIG.SYS, а также от INI-файлов, как от основных файлов конфигурации. На «регистрационную базу» (реестр) была переведена вся конфигурация системы. Основой конфигурации системы стал реестр. Он имел 4 корневых раздела: HKEY_LOCAL_MACHINE, HKEY_CURRENT_USER, HKEY_CLASSES_ROOT и HKEY_USERS.

Реестр стал «сборным»: на диске он хранился в файлах: DEFAULT, SOFTWARE, SYSTEM, а при запуске системы из этих файлов собиралась единая БД.

В комплекте поставки оставался файл REGEDIT.EXE, который по-прежнему позволял просматривать и редактировать только ветку HKEY_CLASSES_ROOT, и появился файл REGEDT32.EXE, который позволял редактировать все ветки реестра.

Далее технология и идеология (назначение) реестра уже не менялись. Все последующие версии Windows (NT 3.5, 95, NT 4.0, 98, 2000, XP, Vista, 7,8) использовали реестр как основную БД, содержащую все основные данные по конфигурации как самой ОС, так и прикладных программ. Далее менялись названия файлов реестра и их расположение, а также название и назначение ключей. --

Современный реестр Windows

Реестр в том виде, как его использует Windows и как видит его пользователь в процессе использования программ работы с реестром, формируется из различных данных. Чтобы получилось то, что видит пользователь, редактируя реестр, происходит следующее.

Вначале, в процессе установки и настройки Windows, на диске формируются файлы, в которых хранится часть данных относительно конфигурации системы.

Затем, в процессе каждой загрузки системы, а также в процессе каждого входа и выхода каждого из пользователей, формируется некая виртуальная сущность, называемая «реестром» — объект REGISTRY\.. Данные для формирования «реестра» частично берутся из тех самых файлов (Software, System ...), частично из информации, собранной ntddetect при загрузке (HKLM\Hardware\Description).

То есть часть данных реестра хранится в файлах, а часть данных формируется в процессе загрузки Windows.

Для редактирования, просмотра и изучения реестра стандартными средствами Windows (программы regedit.exe и regedt32.exe) доступны именно ветки реестра. После редактирования реестра и/или внесения в него изменений эти изменения сразу записываются в файлы.

Однако, есть программы сторонних разработчиков, которые позволяют работать непосредственно с файлами.

Программы оптимизации реестра, твикеры, а также инсталляторы и деинсталляторы программ работают через специальные функции работы с реестром.

Файлы реестра (Хранение данных реестра)

Windows 95/98

- User.dat

- System.dat

Windows ME

- Classes.dat
- User.dat

Windows 2000

Windows XP

Windows Vista

В Windows Vista файлы реестра хранятся там же, где и в Windows XP.

Windows 7

В Windows 7, согласно сведениям из HKEY_LOCAL_MACHINE\SYSTEM\CurrentControlSet\Control\hivelist файлы реестра хранятся в следующих местах:

- 01= Ветка реестра «HKEY_LOCAL_MACHINE\HARDWARE» формируется в зависимости от оборудования (динамически);
- 02= Ветка реестра «HKEY_LOCAL_MACHINE\BCD00000000» формируется из файла «%SystemRoot%\Boot\BCD»
- 03= Ветка реестра «HKEY_LOCAL_MACHINE\SYSTEM» формируется из файла «%SystemRoot%\System32\config\SYSTEM»
- 04= Ветка реестра «HKEY_LOCAL_MACHINE\SOFTWARE» формируется из файла «%SystemRoot%\System32\config\SOFTWARE»
- 05= Ветка реестра «HKEY_LOCAL_MACHINE\SECURITY» формируется из файла «%SystemRoot%\System32\config\SECURITY»
- 06= Ветка реестра «HKEY_LOCAL_MACHINE\SAM» формируется из файла «%SystemRoot%\System32\config\SAM»
- 07= Ветка реестра «HKEY_USERS\DEFAULT» формируется из файла «%SystemRoot%\System32\config\DEFAULT»
- 08= Ветка реестра «HKEY_USERS\S-1-5-18» формируется из файла «%SystemRoot%\System32\config\systemprofile\NTUSER.DAT» (относится к учетной записи system)
- 09= Ветка реестра «HKEY_USERS\S-1-5-19» формируется из файла «%SystemRoot%\ServiceProfiles\LocalService\NTUSER.DAT» (относится к учетной записи LocalService)
- 10= Ветка реестра «HKEY_USERS\S-1-5-20» формируется из файла «%SystemRoot%\ServiceProfiles\NetworkService\NTUSER.DAT» (относится к учетной записи NetworkService)
- 11= Ветка реестра «HKEY_USERS\<SID_пользователя>» формируется из файла «%USERPROFILE%\NTUSER.DAT»
- 12= Ветка реестра «HKEY_USERS\<SID_пользователя>_Classes» формируется из файла «%USERPROFILE%\AppData\Local\Microsoft\Windows\UsrClass.dat»

Резервные копии файлов реестра DEFAULT, SAM, SECURITY, SOFTWARE и SYSTEM находятся в папке «%SystemRoot%\System32\config\RegBack». Само резервное копирование производится силами Планировщика задач в 0 ч. 00 мин. каждые 10 дней по заданию «RegIdleBackup», расположенному в иерархии задач по пути «\Microsoft\Windows\Registry».

Куст реестра - это группа разделов, подразделов и параметров реестра с набором вспомогательных файлов, содержащих резервные копии этих данных. Вспомогательные файлы для всех кустов за исключением HKEY_CURRENT_USER хранятся в системах Windows NT 4.0, Windows 2000, Windows XP, Windows Server 2003 и Windows Vista в папке %SystemRoot%\System32\Config. Вспомогательные файлы для куста HKEY_CURRENT_USER хранятся в папке %SystemRoot%\Profiles\Имя_пользователя. Расширения имен файлов в этих папках указывают на тип содержащихся в них данных. Отсутствие расширения также иногда может указывать на тип содержащихся в файле данных.

Куст реестра	Вспомогательные файлы
HKEY_LOCAL_MACHINE\SAM	Sam, Sam.log, Sam.sav
HKEY_LOCAL_MACHINE\Security	Security, Security.log, Security.sav
HKEY_LOCAL_MACHINE\Software	Software, Software.log, Software.sav
HKEY_LOCAL_MACHINE\System	System, System.alt, System.log, System.sav
HKEY_CURRENT_CONFIG	System, System.alt, System.log, System.sav, Ntuser.dat, Ntuser.dat.log
HKEY_USERS\DEFAULT	Default, Default.log, Default.sav

В Windows 98 файлы реестра называются User.dat и System.dat. В Windows Millennium Edition — Classes.dat, User.dat и System.dat.

Примечание. Средства безопасности в Windows NT, Windows 2000, Windows XP, Windows Server 2003 и Windows Vista позволяют администратору контролировать доступ к разделам реестра.

Таблица содержит перечень и краткое описание стандартных разделов. Максимальная длина имени раздела составляет 255 символов.

Папка/стандартный раздел	Описание
HKEY_CURRENT_USER	Данный раздел является корневым для данных конфигурации пользователя, вошедшего в систему в настоящий момент. Здесь хранятся папки пользователя, цвета экрана и параметры панели управления. Эти сведения сопоставлены с профилем пользователя. Вместо полного имени раздела иногда используется аббревиатура HKCU.
HKEY_USERS	Данный раздел содержит все активные загруженные профили пользователей компьютера. Раздел

	HKEY_CURRENT_USER является подразделом раздела HKEY_USERS. Вместо полного имени раздела иногда используется аббревиатура HKU.
HKEY_LOCAL_MACHINE	Раздел содержит параметры конфигурации, относящиеся к данному компьютеру (для всех пользователей). Вместо полного имени раздела иногда используется аббревиатура HKLM.
HKEY_CLASSES_ROOT	Является подразделом HKEY_LOCAL_MACHINE\Software. Хранящиеся здесь сведения обеспечивают выполнение необходимой программы при открытии файла с использованием проводника. Вместо полного имени раздела иногда используется аббревиатура HKCR. Начиная с Windows 2000, эти сведения хранятся как в HKEY_LOCAL_MACHINE, так и в HKEY_CURRENT_USER. Раздел HKEY_LOCAL_MACHINE\Software\Classes содержит параметры по умолчанию, которые относятся ко всем пользователям локального компьютера. Параметры, содержащиеся в разделе HKEY_CURRENT_USER\Software\Classes, переопределяют принятые по умолчанию и относятся только к текущему пользователю. Раздел HKEY_CLASSES_ROOT включает в себя данные из обоих источников. Кроме того, раздел HKEY_CLASSES_ROOT предоставляет эти объединенные данные программам, разработанным для более ранних версий Windows. Изменения настроек текущего пользователя выполняются в разделе HKEY_CURRENT_USER\Software\Classes. Модификация параметров по умолчанию должна производиться в разделе HKEY_LOCAL_MACHINE\Software\Classes. Данные из разделов, добавленных в HKEY_CLASSES_ROOT, будут сохранены системой в разделе HKEY_LOCAL_MACHINE\Software\Classes. Если изменяется параметр в одном из подразделов раздела HKEY_CLASSES_ROOT и такой подраздел уже существует в HKEY_CURRENT_USER\Software\Classes, то для хранения информации будет использован раздел HKEY_CURRENT_USER\Software\Classes, а не HKEY_LOCAL_MACHINE\Software\Classes.
HKEY_CURRENT_CONFIG	Данный раздел содержит сведения о профиле оборудования, используемом локальным компьютером при запуске системы.

Примечание. Реестр 64-разрядных версий Windows XP и Windows Server 2003 и Windows Vista подразделяется на 32- и 64-разрядные разделы. Большинство 32-разрядных разделов имеют те же имена, что и их аналоги в 64-разрядном разделе, и наоборот. По умолчанию редактор реестра 64-разрядных версий Windows XP и Windows Server 2003 и Windows Vista отображает 32-разрядные разделы в следующем узле:

HKEY_LOCAL_MACHINE\Software\WOW6432Node

Следующая таблица содержит список типов данных, определенных и используемых Windows на сегодняшний день. Максимальная длина имени параметра:

- Windows Server 2003, Windows XP и Windows Vista: 16 383 символов
- Windows 2000: 260 символов ANSI или 16 383 символа Юникод

- Windows 95, Windows 98 и Windows Millennium Edition: 255 символов

Значения большого размера (больше 2048 байт) хранятся во внешних файлах, а в реестр заносится имя такого файла. Это способствует повышению эффективности использования реестра.

Максимальный размер параметра:

- Windows NT 4.0/Windows 2000/Windows XP/Windows Server 2003/Windows Vista:

Доступная память

- Windows 95, Windows 98 и Windows Millennium Edition: 16 300 байт

Примечание. Общий размер всех параметров раздела не должен превышать 64 КБ.

Имя	Тип	Описание
Двоичный параметр	REG_BINARY	Необработанные двоичные данные. Большинство сведений об аппаратных компонентах хранится в виде двоичных данных и выводится в редакторе реестра в шестнадцатеричном формате.
Параметр DWORD	REG_DWORD	Данные представлены в виде значения, длина которого составляет 4 байта (32-разрядное целое). Этот тип данных используется для хранения параметров драйверов устройств и служб. Значение отображается в окне редактора реестра в двоичном, шестнадцатеричном или десятичном формате. Эквивалентами типа DWORD являются DWORD_LITTLE_ENDIAN (самый младший байт хранится в памяти в первом числе) и REG_DWORD_BIG_ENDIAN (самый младший байт хранится в памяти в последнем числе).
Расширяемая строка данных	REG_EXPAND_SZ	Строка данных переменной длины. Этот тип данных включает переменные, обрабатываемые при использовании данных программой или службой.
Многострочный параметр	REG_MULTI_SZ	Многострочный текст. Этот тип, как правило, имеют списки и другие записи в формате, удобном для чтения. Записи разделяются пробелами, запятыми или другими символами.
Строковый параметр	REG_SZ	Текстовая строка фиксированной длины.
Двоичный параметр	REG_RESOURCE_LIST	Последовательность вложенных массивов. Служит для хранения списка ресурсов, которые используются драйвером устройства или управляемым им физическим устройством. Обнаруженные данные система сохраняет в разделе \ResourceMap. В окне редактора реестра эти данные отображаются в виде двоичного параметра в шестнадцатеричном формате.

Двоичный параметр	REG_RESOURCE_REQUIREMENTS_LIST	Последовательность вложенных массивов. Служит для хранения списка драйверов аппаратных ресурсов, которые могут быть использованы определенным драйвером устройства или управляемым им физическим устройством. Часть этого списка система записывает в раздел \ResourceMap. Данные определяются системой. В окне редактора реестра они отображаются в виде двоичного параметра в шестнадцатеричном формате.
Двоичный параметр	REG_FULL_RESOURCE_DESCRIPTOR	Последовательность вложенных массивов. Служит для хранения списка ресурсов, которые используются физическим устройством. Обнаруженные данные система сохраняет в разделе \HardwareDescription. В окне редактора реестра эти данные отображаются в виде двоичного параметра в шестнадцатеричном формате.
Отсутствует	REG_NONE	Данные, не имеющие определенного типа. Такие данные записываются в реестр системой или приложением. В окне редактора реестра отображаются в виде двоичного параметра в шестнадцатеричном формате.
Ссылка	REG_LINK	Символическая ссылка в формате Юникод.
Параметр QWORD	REG_QWORD	Данные, представленные в виде 64-разрядного целого. Начиная с Windows 2000, такие данные отображаются в окне редактора реестра в виде двоичного параметра.

Использование редактора реестра

Предупреждение. Неправильное изменение параметров системного реестра с помощью редактора реестра или любым иным способом может привести к серьезным неполадкам. Для их устранения может потребоваться переустановка операционной системы. Корпорация Майкрософт не гарантирует, что эти неполадки можно будет устранить. Ответственность за изменение реестра несет пользователь.

Редактор реестра можно использовать для выполнения следующих задач:

- поиск поддерева, раздела, подраздела или параметра;
- добавление подраздела или параметра;
- изменение значения параметра;
- удаление подраздела или параметра;
- переименование подраздела или параметра.

Область переходов редактора реестра отображает набор папок. Каждая папка представляет собой раздел реестра локального компьютера. При просмотре реестра удаленного компьютера будут видны только два стандартных раздела: HKEY_USERS и HKEY_LOCAL_MACHINE.

Использование групповой политики

Консоль управления Microsoft (MMC) содержит средства администрирования, которые используются для управления сетями, компьютерами, службами и другими системными компонентами. С помощью оснастки "Групповая политика" администратор может определить параметры безопасности для пользователей и компьютеров. Групповую политику можно реализовать на локальном компьютере с помощью локальной оснастки "Групповая политика" (файл Gpedit.msc) или в Active Directory с помощью оснастки "Active Directory - пользователи и компьютеры". Дополнительные сведения об использовании групповой политики см. в справке по соответствующей оснастке, используемой для работы с групповой политикой.

Использование файла реестра (REG)

Для внесения изменений в реестр можно создать файл реестра (с расширением REG) и выполнить его на соответствующем компьютере. Выполнить REG-файл можно вручную или с помощью сценария входа. Дополнительные сведения см. в следующей статье базы знаний Майкрософт:

Использование сервера сценариев Windows

Сервер сценариев Windows позволяет выполнять сценарии на языке VBScript или JScript непосредственно в операционной системе. В таких сценариях для удаления, чтения и записи разделов и параметров реестра используются методы сервера сценариев Windows.

Использование инструментария управления Windows

Инструментарий управления Windows (WMI) - это компонент операционной системы Windows, который представляет собой систему управления предприятием через Интернет (WBEM) в реализации корпорации Майкрософт. WBEM - это отраслевая инициатива по разработке стандартной технологии доступа к данным, необходимым для управления средой предприятия. Инструментарий WMI позволяет автоматизировать административные задачи (включая изменение реестра) в среде предприятия. Его можно использовать в языках сценариев, которые имеют обработчик в Windows и работают с объектами Microsoft ActiveX. Кроме того, для изменения реестра Windows можно использовать программу командной строки Wmic.exe из состава инструментария управления Windows.

Использование консольной программы редактирования реестра Windows

Для редактирования системного реестра можно воспользоваться консольной программой редактирования реестра Windows (Reg.exe). Для получения справки по программе Reg.exe введите в командной строке команду **reg /?** и нажмите кнопку **ОК**.

Восстановление реестра

Для восстановления реестра используйте подходящий способ из числа указанных ниже.

Восстановление разделов реестра

Чтобы восстановить экспортированные разделы реестра, необходимо выполнить REG-файл, сохраненный при экспорте подразделов реестра. Кроме того, можно восстановить весь реестр из его резервной копии. Дополнительные сведения о восстановлении всего реестра см. в подразделе "Восстановление всего реестра" этой статьи.

Восстановление всего реестра

Для восстановления всего реестра необходимо восстановить состояние системы из его резервной копии. Дополнительные сведения о восстановлении состояния системы из резервной копии см. в следующей статье базы знаний Майкрософт:

Примечание. При создании резервной копии состояния системы обновленные копии файлов реестра сохраняются в папке %SystemRoot%\Repair. Если не удастся запустить Windows XP после внесения изменений в реестр, воспользуйтесь инструкциями из первой части указанной ниже статьи базы знаний Майкрософт для подстановки сохраненных файлов реестра:

5. Настройка конфигурации аппаратных средств. Профили оборудования. Ресурсы ОС. Управление продуктивностью.

Системные параметры. Повышение производительности

5.1. Повышение производительности

Мы поговорим о параметрах реестра, позволяющих повысить производительность системы, а также о других параметрах, связанных с тонкой настройкой системы. Для вступления в силу большинства параметров, описанных в этой главе, необходима перезагрузка компьютера.

5.1.1. Ускорение работы с памятью

В реестре Windows (данный трюк подходит как для Windows XP, так и для Windows Vista/7) есть несколько параметров, влияющих на управление памятью. Изменение этих параметров может существенно повысить производительность, может несущественно повысить производительность, может понизить производительность системы, а может вообще сделать систему неработоспособной. Поэтому относитесь к этим параметрам с особой осторожностью и внимательно читайте текст данной главы. И еще: упомянутые здесь параметры следует изменять, только если у вас большой объем оперативной памяти. Чтобы получить должный эффект, вам необходимо иметь более 512 Мбайт RAM для Windows XP и более 1 Гбайт для Windows Vista/7. Параметры управления памятью находятся в разделе HKLM\SYSTEM\CurrentControlSet\Control\Session Manager\Memory Management.

Вот эти параметры:

- DisablePagingExecutive (тип данных REG_DWORD) — по умолчанию выключен и его значение равно 0. Если его включить (присвоить значение 1), для пользователей система не будет записывать на диск при подкачке коды ядра и драйверов, а всегда будет держать их в оперативной памяти. Приложения будут быстрее реагировать на действия пользователей, поскольку не нужно будет загружать нужный приложению код ядра из файла подкачки. Но поскольку коды ядра и драйверов будут постоянно храниться в оперативной памяти, нужен большой объем физически установленной RAM;
- LargeSystemCache (тип данных REG_DWORD) — по умолчанию также выключен и его значение равно 0. Если включить этот параметр (присвоить значение 1), то ядро системы будет постоянно находиться в памяти, что ускорит доступ к ядру и повысит производительность всей системы. Этот параметр подойдет для машины, используемой в качестве сервера, а не для обычной рабочей станции или домашнего компьютера.

5.1.2. Выгрузка из памяти неиспользуемых DLL

Windows продолжает хранить в памяти однажды загруженные DLL, даже если они уже не используются программами. Это делается для ускорения доступа к библиотекам, но не очень экономно расходует оперативную память. Вы можете добавить параметр реестра, выгружающий неиспользуемые библиотеки из памяти. Если производительность системы, наоборот, понизится, его лучше удалить. Параметр называется AlwaysUnloadDll, имеет тип REG_DWORD и находится в разделе HKLM\SOFTWARE\Microsoft\Windows\CurrentVersion\Explorer. Параметру нужно присвоить значение 1.

5.1.3. Автоматическое очищение файла подкачки

Вы можете включить параметр очистки файла подкачки (pagefile.sys) при перезагрузке (завершении работы) системы. С одной стороны, это позволит немного ускорить загрузку системы, но в то же время вы потеряете на времени завершения работы — компьютер будет выключаться медленнее. С другой стороны, в файле подкачки могут находиться конфиденциальные данные, в том числе и пароль в открытом виде, несмотря на очень тщательное шифрование файла с паролем. Поэтому файл подкачки по завершении работы лучше очищать. Перейдите в раздел HKLM\SYSTEM\CurrentControlSet\Control\Session Manager \Memory Management и установите значение 1 для параметра ClearPageFileAtShutdown.

5.1.4. Повышение производительности системы путем запрета выгрузки драйверов

Если у вас достаточно оперативной памяти (хотя бы 2 Гбайт), можно увеличить производительность системы, запретив выгружать из оперативной памяти на жесткий диск код драйверов и другой служебный, но не используемый в данное время код. Перейдите в раздел HKLM\SYSTEM\CurrentControlSet\Control\Session Manager\Memory Management и создайте параметр DisablePagingExecutive (тип данных — REG_DWORD) со значением 1.

5.1.5. Ускорение завершения работы системы

При завершении работы помимо всего прочего система завершает запущенные сервисы (службы). Если сервис не отвечает, то система выжидает некоторое время, а потом завершает службу. Ускорить завершение работы можно путем уменьшения времени ожидания. Создайте в разделах реестра HKLM\SYSTEM\CurrentControlSet\Control, HKLM\SYSTEM\CurrentControlSet001\Control и HKLM\SYSTEM\CurrentControlSet002\Control строковый (REG_SZ) параметр WaitToKillServiceTimeout. Его значение — это время в миллисекундах, которое система будет ждать перед завершением службы. По умолчанию система ждет 20 секунд, т. е. 20 000 миллисекунд. Это слишком долго, вполне достаточно 10 секунд, т. е. 10 000 миллисекунд.

5.1.6. Отключение планировщика Windows

В Windows есть специальная программа — планировщик, выполняющая задания (другие программы) в определенное время. Обычно планировщиком никто никогда не пользуется, а поэтому его можно отключить, что сократит время загрузки Windows1. Чтобы отключить планировщик, перейдите в раздел HKLM\SYSTEM\CurrentControlSet\Services\Schedule, найдите параметр Start (тип данных — REG_DWORD) и установите для него значение 0. Чтобы вернуть все в исходное состояние, нужно установить значение 2.

На самом деле от данного сервиса в Windows Vista и Windows 7 зависят такие важные системные задачи, как, например, резервное копирование, создание точек восстановления системы, регулярный запуск Windows Defender, сканирование системы антивирусными программами по расписанию и др. Сразу после блокирования сервиса ничего ужасного, конечно, не произойдет, зато потом могут начаться проблемы. А выигрыш по скорости загрузки будет ничтожным.

5.1.7. Увеличение производительности NTFS

Можно долго спорить о том, какая файловая система лучше — FAT32 или NTFS. NTFS обеспечивает должный уровень безопасности и предоставляет возможности, которые не доступны в FAT32, кроме того, она поддерживает файлы больших размеров. В Windows XP максимальный размер файла для FAT32 — 4 Гбайт. А что делать, если вам нужно создать файл большего размера? Ведь рано или поздно вам придется создать образ DVD, а это уже 4,5 Гбайт! Хотя у FAT32 также есть свои преимущества — она работает быстрее, чем NTFS. NTFS медленнее, чем FAT32, только потому, что: при каждом обращении к файлу или каталогу ей приходится обновлять метку последнего доступа. При большом количестве файлов или каталогов это снижает производительность системы; для совместимости со старыми приложениями в NTFS-разделе создается специальная таблица файлов, содержащая имена файлов в формате MSDOS (как известно, это 8 символов для имени и 3 — для расширения файла). Для повышения производительности NTFS нужно перейти в раздел HKLM\SYSTEM\CurrentControlSet\Control\FileSystem и установить значение 1 для следующих параметров: NtfsDisableLastAccessUpdate; NtfsDisable8dot3NameCreation. Первый параметр отключает запись последнего времени доступа, а второй — создание таблицы для совместимости со старыми приложениями. Для большей производительности можно дополнительно включить параметр NtfsDisableEncryption, но с точки зрения безопасности, этого не следует делать, потому что он отключает шифрование данных, обеспечиваемое файловой системой NTFS.

Многие пользователи (особенно те, кто создает мультизагрузочные системы) действительно продолжают пользоваться FAT32, Microsoft, со всей очевидностью, уже считает эту тему закрытой. Уже сейчас FAT32 поддерживается исключительно для обратной совместимости, а также всячески ущемляется и дискриминируется. Она даже не поддерживается средствами резервного

копирования и восстановления, встроенными в Windows Vista и Windows 7, о чем не мешает помнить поклонникам мультзагрузки.

5.1.5. Включить поддержку UDMA-66 на чипсетах Intel

Внимание: данный трюк предназначен только для материнских плат, основанных на чипсетах Intel, и жестких дисков IDE (не SATA). Убедитесь также, что ваш жесткий диск поддерживает UDMA-66 и подключен к материнской плате с помощью кабеля 80-pin. Если параметры вашего оборудования отвечают указанным требованиям, создайте в разделе HKLM\System\CurrentControlSet\Control\Class\{4D36E96AE325-11CE-BFC1-08002BE10318}\0000 параметр с типом данных REG_DWORD и именем EnableUDMA66. Присвойте ему значение 1.

5.1.9. Отключаем неиспользуемые сервисы

5.1.9.1. Зачем нужно отключать лишние сервисы?

Служба (или сервис, от англ. service) — это специальная программа, выполняющаяся в фоновом режиме и не имеющая пользовательского интерфейса. В большинстве случаев каждая служба представляет собой важный компонент операционной системы, без которого она не может функционировать. Значительно реже свои собственные службы добавляют программы сторонних разработчиков, например, Антивирус Касперского, Outpost Security Suite и другие программы. Некоторые из служб можно отключить, однако к этому процессу следует подходить с осторожностью, чтобы случайно не отключить важную системную службу.

Каждая служба, даже если она просто находится в памяти и ничего не делает, занимает оперативную память и процессорное время. Следовательно, при отключении неиспользуемых служб мы оптимизируем работу оперативной памяти и процессорного времени, благодаря чему наш компьютер будет работать быстрее. Однако не нужно думать, что если вы отключите одну службу, то ваш компьютер сразу же начнет "летать". Одна служба ничего не решит. Нужно подойти к процессу отключения служб комплексно. Надо понимать, что повышение производительности — далеко не единственная причина отключения неиспользуемых служб. Некоторые службы потенциально опасны, т. е. могут использоваться для атаки вашего компьютера для пользователей злоумышленником. Конечно, если установлен брандмауэр, то ваш компьютер находится в относительной безопасности, а вот если вы заблокировали встроенный брандмауэр и не используете никакого стороннего, то желательно отключить некоторые службы. Итак, рассмотрим отключение неиспользуемых служб, как средство повышения производительности компьютера и его защиты от атак.

5.1.9.2. Как отключить сервис?

Для управления службами используется специальная программа, для запуска которой нужно нажать кнопку Пуск (Start) и ввести в поле поиска, расположенное в нижней части меню, команду services.msc.

5.1. Программа управления службами в Windows 7

В колонке Имя (Name) приводится название службы, в колонке Описание (Description) — описание функций, для выполнения которых она предназначена. Для более удобного просмотра описания можно щелкнуть по имени службы, а потом прочитать его в левой части окна. Колонка Состояние (Status) отображает текущее состояние службы: если она работает, то вы увидите соответствующее сообщение. Глава

5. Системные параметры. Повышение производительности 105

Колонка Тип запуска (Startup Type) отображает способ запуска службы:

- Автоматически (Auto) — служба запускается автоматически при запуске Windows;
- Автоматически (отложенный запуск) (Automatic (Delayed Start)) — новый вид запуска служб, появившийся, начиная с Windows Vista, и, безусловно, применяющийся и в Windows 7. Он используется для тех сервисов, которые выполняют важные функции, но не нужны пользователю немедленно после регистрации в системе. Такие сервисы запускаются чуть позднее, и за счет этого система будет быстрее готова к регистрации пользователя;

- Вручную (Manual) — служба не запускается автоматически при запуске Windows, однако может быть запущена вручную пользователем или другой службой (т. е. это так называемый запуск по требованию);
- Отключена (Disabled) — служба вообще не запускается. Если данная служба понадобится, то сначала придется изменить тип запуска на автоматический, автоматический с отложенным запуском или ручной.

Для изменения типа запуска дважды щелкните по строке с нужной службой и выберите требуемый тип запуска, например, Отключена (Disabled) для полного отключения. Если служба запущена, то будет активна кнопка Стоп (Stop), которая используется для остановки службы. Останавливать службу перед ее отключением не нужно — Windows это сделает автоматически. Для каждой службы приводится ее довольно подробное описание. Если вы знаете, что служба вам не нужна, ее можно отключить. Если вы не уверены, можно найти дополнительную информацию о службе в Интернете (поиск Гуглом еще никто не отменял). Лучше всего отключать службы поэтапно: отключите некоторые из них, запомните, что именно вы отключили. Затем немного поработайте, чтобы убедиться, что система работает корректно. Если какие-то нужные вам функции оказались недоступны, вновь включите ранее отключенные вами службы.

5.2. Настройка автозапуска программ

Для автоматического запуска программ используются следующие разделы реестра: HKCU\Software\Microsoft\Windows\CurrentVersion\Run; HKCU\Software\Microsoft\Windows\CurrentVersion\Runonce; HKLM\Software\Microsoft\Windows\CurrentVersion\Run; HKLM\Software\Microsoft\Windows\CurrentVersion\Runonce; HKLM\SOFTWARE\Microsoft\Windows\CurrentVersion\RunOnceEx. Как мы уже говорили, разделы в HKCU содержат настройки для текущего пользователя, а в HKLM — для всех пользователей системы. В разделы Run включены списки программ, которые автоматически запускаются при каждом входе пользователя в систему. В отличие от него, программы, содержащиеся в разделах Runonce, будут запущены только один раз при входе пользователя в систему, после чего этот список будет очищен. Раздел RunonceEx аналогичен Runonce с тем отличием, что программы из их списков будут выполнены один раз при загрузке системы, а не при входе определенного пользователя. Теперь о том, как формируются списки автозапуска. Каждый список — это набор параметров типа REG_SZ. Имя параметра произвольное, а его значение — команда, которую нужно выполнить.

Вот, например, очень хороший и заслуживающий доверия источник информации о службах Windows всех версий — начиная с Windows 2000 и заканчивая Windows 7: <http://www.blackviper.com/>.

5.3. Список автозапуска

Для добавления программы в список автозапуска нужно создать параметр типа REG_SZ, содержащий команду для запуска программы. Чтобы удалить программу из списка автозапуска, достаточно удалить соответствующий ей параметр из раздела (или разделов) Run*. Для управления автозапуском также используются следующие параметры:

HKEY_LOCAL_MACHINE\SOFTWARE\Microsoft\Windows\CurrentVersion\policies \Explorer\ — параметр DisableCurrentUserRun — если параметр включен (его значение равно 1), то пользовательский список Run из HKCU не будет выполнен;

HKEY_LOCAL_MACHINE\SOFTWARE\Microsoft\Windows\CurrentVersion \policies\Explorer\ — параметр DisableCurrentUserRunOnce — отключает пользовательский список RunOnce из HKCU;

HKEY_LOCAL_MACHINE\SOFTWARE\Microsoft\Windows\CurrentVersion \policies\Explorer\ — параметр DisableLocalMachineRun — отключает "общий" список автозапуска Run из HKLM;

HKEY_LOCAL_MACHINE\SOFTWARE\Microsoft\Windows\CurrentVersion\policies \Explorer\ — DisableLocalMachineRunOnce — отключает "общий" список автозапуска RunOnce из HKLM.

5.3. Удаление программ из списка установленных (Uninstall своими руками)

Для удаления сведений об установке программы из реестра перейдите в раздел реестра HKLM\Software\Microsoft\Windows\CurrentVersion\Uninstall. В нем будут подразделы с именами, содержащими цифры и буквы, например, {01B28B7B-EEC6-12D5-5B5A-5A7EBDF5EFBA},

5.4. Сведения об установленных программах

Каждый такой раздел соответствует какой-то программе. Какой именно? Имя программы содержится в параметре DisplayName. Например, у меня раздел HKEY_LOCAL_MACHINE\SOFTWARE\Microsoft\Windows\CurrentVersion\Uninstall\ {C427E746-4EC9-4E3C-AACB-C6BB1F714D7F} соответствует программе Uniblue DriverScanner 2009. Перед удалением раздела рекомендуется экспортировать этот раздел в REG-файл, чтобы в случае чего у вас была возможность его восстановить. Как только раздел будет экспортирован, можно его удалить. Глава

5.4. Что делать с зависшими программами?

Иногда программы зависают, и их невозможно закрыть обычным образом. Тогда приходится открывать Диспетчер задач (Windows Task Manager), искать зависший процесс в числе работающих и вручную его завершать. Запустить Диспетчер задач (Windows Task Manager) проще всего одновременным нажатием клавиш <Ctrl>+<Shift>+<Esc>. Можно настроить Windows и таким образом, чтобы она автоматически завершала зависшие процессы. Для этого перейдите в раздел HKCU\Control Panel\Desktop.

В этом разделе вы найдете следующие параметры:

- AutoEndTasks (REG_DWORD) — если присвоить этому параметру значение 1, то Windows будет автоматически завершать зависшие задачи;
- HungAppTimeout (REG_SZ) — период, по прошествии которого можно считать приложение зависшим. Время отсчитывается с момента, когда приложение перестало отвечать на запросы операционной системы. По умолчанию оно равно 5000 мс или 5 с;
- WaitToKillAppTimeout (REG_SZ) — период ожидания перед завершением процесса (вдруг он "одумается"). По умолчанию это значение составляет 20 000 мс или 20 с. Сложив значения параметров HungAppTimeout и WaitToKillAppTimeout, можно заметить, что по умолчанию Windows понадобится 25 с, чтобы завершить процесс. А теперь немного практики. Чаще всего приложения зависают, ожидая ответа от какого-нибудь устройства или другого процесса. При этом бывает и так, что ожидаемое приложение не отзывается из-за большой загруженности процессора. Срок продолжительностью 5 секунд недостаточен для того, чтобы сделать вывод о том, что программа зависла. Нужно увеличить значение параметра HungAppTimeout до 10 000, т. е. 10 с. Если прошло 10 секунд, и нужное приложение не отзывается на запросы системы, его можно смело завершать. Вообще говоря, можно задать для параметра WaitToKillAppTimeout значение 0, но лучше все-таки немного подождать, хотя бы 5 секунд, т. е. 5000 мс. В целом, Windows 7 довольно хорошо справляется с зависшими программами, но на всякий случай лучше знать, где находятся параметры аварийного завершения программ. До недавнего времени я тоже думал, что Windows 7 неуязвима, но вчера попытался подключить свой телефон, чтобы скачать с него фотографии. Фотографии были успешно скачаны, но, когда я отключил телефон от компьютера, увидел синий экран смерти. В Windows 7 я его до этого не видел и искренне надеялся, что и не увижу. 110 Часть I. Для пользователей

5.5. Служба SuperFetch

Служба SuperFetch позволяет ускорить выполнение программ, с которыми вы часто работаете. Служба работает так: она помещает файлы программ, которые вы часто используете. За счет этого достигается ускорение запуска программ, ведь практически все необходимые файлы уже загружены службой SuperFetch. Служба ведет себя довольно интеллектуально и запоминает, какие программы и когда вы запускаете. Скажем, если вы на выходных играете в Diablo II, а в будние дни преимущественно работаете с офисными приложениями, то файлы игры не будут загружены в будни, ровно как и файлы офисных приложений не будут загружены на выходных. Так достигается

экономия оперативной памяти (понятно, что если загрузить в нее сразу все программы, с которыми вы работаете, то производительность будет оставлять желать лучшего). Если оперативной памяти в компьютере мало (скажем, 1 Гбайт или меньше), то служба для ускорения работы системы может использовать флэш-память. Да, она медленнее оперативной памяти, но быстрее жестких дисков (правда, не все модели, а те, которые поддерживают технологию Windows ReadyBoost). Чтобы служба SuperFetch использовала флэш-носитель, подключите флэшку к компьютеру, а затем в окне автозапуска (AutoPlay) выберите команду Ускорить работу системы (Speed up my system).

Настройки службы SuperFetch хранятся в разделе HKLM\SYSTEM\CurrentControlSet\Control\Session Manager\Memory Management\PrefetchParameters.

В этом разделе вы найдете три параметра типа REG_DWORD:

- EnableBootTrace — включает трассировку работы службы SuperFetch, значение по умолчанию — 0, т. е. трассировка выключена. Трассировка нужна только в том случае, если служба работает не так, как должна.
- EnablePrefetcher — определяет, будет ли включен механизм Prefetcher (механизм упреждающей выборки).
- EnableSuperfetch — определяет, будет ли включена служба SuperFetch. Последние два параметра могут принимать четыре значения: 0 — функция выключена; 1 — функция включена, но только для загрузки системы; 2 — функция будет доступна только во время работы системы, но будет отключена при загрузке системы; 3 — функция будет доступна как во время загрузки, так и во время работы системы. Глава

5.6. Уменьшение фрагментации больших файлов Параметр ContigFileAllocSize (тип REG_DWORD) раздела HKLM\System\CurrentControlSet\Control\FileSystem задает максимальный размер нефрагментируемого блока данных (в байтах). Этот параметр используется для того, чтобы система перед записью большого файла нашла сначала для него то место, на котором файл окажется в наименьшей степени фрагментированным. По умолчанию система начинает записывать файл на первый же обнаруженный фрагмент свободного пространства. Записав несколько частей файла, система обнаруживает, что следующие блоки заняты. Затем она начинает искать следующий свободный блок. Вполне может получиться и так, что первая часть файла физически записана в "начале" диска, вторая — в середине, а третья — в конце. Все это замедляет последующую работу системы с этим файлом — снижается скорость его чтения и записи. Когда же система будет последовательно (по возможности) располагать фрагменты файла, это снизит фрагментацию и повысит общую производительность системы. Осталось только подобрать значение параметра ContigFileAllocSize. Для небольших жестких дисков (до 15 Гбайт) нужно установить значение 00000200. Если объем жесткого диска 20–40 Гбайт — 00000400 или 00000600. Для жестких дисков размером 40 Гбайт и выше — 00001000. Экспериментируйте со значением этого параметра, чтобы добиться максимальной производительности.

5.7. Выключение автоматического обновления Windows

По умолчанию Windows обновляет себя, не спрашивая об этом разрешения пользователя. Не верите? Установите брандмауэр Outpost Security Suite Pro и включите контроль компонентов. В среднем 2–3 раза в день вы будете видеть сообщение о том, что компоненты приложений изменены. Иногда приложения обновляют сами себя сами, а иногда "старается" именно служба автоматического обновления Windows.

В разделе HKLM\SOFTWARE\Microsoft\Windows\CurrentVersion\WindowsUpdate\Auto Update находятся параметры автоматического обновления:

- AUOptions (REG_DWORD); AUState (REG_DWORD). Отключить автоматическое обновление можно, присвоив следующие значения этим параметрам: AUOptions = 1;
- AUState = 7. Если вы хотите только получать сообщения о возможности загрузки обновлений, измените данные параметры так: AUOptions = 2; AUState = 2. Если нужно загружать обновления, а потом только уведомлять об их готовности к установке, то установите следующие значения указанных параметров: AUOptions = 3; AUState = 2.

5.5. Установка пути к дистрибутиву Windows

Вы скопировали дистрибутив на жесткий диск, а Windows по-прежнему его ищет на DVD? Измените параметр реестра SourcePath (REG_SZ) в разделе HKLM\SOFTWARE\Microsoft\Windows\CurrentVersion\Setup. В качестве значения этого параметра укажите путь к дистрибутиву Windows.

5.9. Установка пути к каталогу Program Files

Путь к каталогу, в который по умолчанию устанавливаются все программы и по умолчанию называется C:\Program Files, задается строковым параметром ProgramFilesDir в разделе HKLM\SOFTWARE\Microsoft\Windows\CurrentVersion

5.10. Настройка службы времени

Если вы используете службу времени, то можете настроить интервал синхронизации часов компьютера с сервером времени. Для этого перейдите в раздел HKLM\SYSTEM\CurrentControlSet\Services\W32Time\TimeProviders\NtpClient и установите значение параметра SpecialPollInterval (REG_DWORD). Его значение задается в секундах.

5.11. Что делать в случае отказа системы

В случае отказа системы Windows позволяет настроить выполнение следующих действий:

- автоматическая перезагрузка — параметр AutoReboot (REG_DWORD);
- запись события в системный журнал — параметр LogEvent (REG_DWORD);
- отправка административного сообщения — параметр SendAlert (REG_DWORD);
- запись отладочной информации CrashDumpEnabled (REG_DWORD).

Самым полезным является первый параметр, позволяющий перезагружать компьютер в случае отказа Windows. Запись события в системный журнал бессмысленна — от того, что в журнал будет записано сообщение об отказе системы, легче вам не станет, тем более что причина сбоя не указывается. Если хотите знать причину сбоя, то нужно включить последний параметр — запись дампа памяти, но чтобы понять причину по дампу памяти, нужно быть настоящим гением. Отправка административного сообщения тоже не нужна. Итак, включим автоматическую перезагрузку компьютера в случае сбоя. Для этого перейдите в раздел HKLM\SYSTEM\CurrentControlSet\Control\CrashControl и присвойте параметру AutoReboot значение 1.

5.12. Исправление ошибки инсталлятора в Windows 7

Некоторые приложения невозможно установить в Windows 7 — их установка приводит к краху инсталлятора Windows. Если при установке программы произошла ошибка (имеются в виду только MSI-инсталляторы), запустите редактор реестра и удалите раздел HKLM\SOFTWARE\Microsoft\SQMClient\Windows\DisabledSessions. После этого повторите попытку установки программы.

5.13. Комплексная доработка Windows 7

Рассмотрим REG-файл, выполняющий следующие действия:

- отключает уведомление о нехватке дискового пространства;
- ускоряет работу IE 8 путем отключения поиска сетевых принтеров и сетевых заданий;
- добавляет команды "Move To" (Переместить в) и "Copy To" (Копировать в) в контекстное меню;
- уменьшает время открытия меню;
- ускоряет завершение зависших процессов;
- добавляет команду "Take Ownership" (Изменить владельца) в контекстное меню каталога, что позволит быстро изменить владельца каталога.

Содержимое REG-файла приведено в листинге 5.1.

Листинг 5.1. Шесть трюков реестра в одном REG-файле Windows Registry Editor Version 5.00

```
[HKEY_CURRENT_USER\Software\Microsoft\Windows\CurrentVersion\Policies\Explorer]
```

```
"NoLowDiskSpaceChecks"=dword:00000001
```

```
"LinkResolveIgnoreLinkInfo"=dword:00000001
```

```
"NoResolveSearch"=dword:00000001
```

"NoResolveTrack"=dword:00000001
"NoInternetOpenWith"=dword:00000001

[HKEY_CLASSES_ROOT\AllFilesystemObjects\shellex\ContextMenuHandlers\Copy To]
@="{C2FBB630-2971-11D1-A18C-00C04FD75D13}"

[HKEY_CLASSES_ROOT\AllFilesystemObjects\shellex\ContextMenuHandlers\Move To]
@="{C2FBB631-2971-11D1-A18C-00C04FD75D13}"

[HKEY_CURRENT_USER\Control Panel\Desktop]
"AutoEndTasks"="1"
"HungAppTimeout"="1000"
"MenuShowDelay"="8"
"WaitToKillAppTimeout"="2000"
"LowLevelHooksTimeout"="1000"

[HKEY_LOCAL_MACHINE\SYSTEM\CurrentControlSet\Control]
"WaitToKillServiceTimeout"="1000"

[-HKEY_LOCAL_MACHINE\SOFTWARE\Microsoft\Windows\CurrentVersion\Explorer\RemoteComputer]

[HKEY_CLASSES_ROOT*\shell\runas]
@="Take Ownership"
"NoWorkingDirectory"=""

[HKEY_CLASSES_ROOT*\shell\runas\command]
@="cmd.exe /c takeown /f \"%1\" && icacs \"%1\" /grant administrators:F"
"IsolatedCommand"="cmd.exe /c takeown /f \"%1\" && icacs \"%1\" /grant administrators:F"

[HKEY_CLASSES_ROOT\Directory\shell\runas]
@="Take Ownership"
"NoWorkingDirectory"=""

[HKEY_CLASSES_ROOT\Directory\shell\runas\command]
@="cmd.exe /c takeown /f \"%1\" /r /d y &&
"IsolatedCommand"="cmd.exe /c takeown /f \"%1\" /r /d y && icacs \"%1\" /grant administrators:F /t"

Вопросы.